

NHAI DATA LAKE 7.0

Secure Offline Facial Recognition & Liveness Detection System for Remote
Infrastructure Locations

TECHNICAL PROPOSAL

PROBLEM STATEMENT & OBJECTIVE

⚠️ The Problem

How can we accurately authenticate personnel in zero-network zones using mid-range devices without high-end GPUs, while keeping the AI model footprint under 20MB and latency <1s?

🎯 The Objective

Development of a highly accurate, lightweight, entirely offline facial recognition and liveness detection algorithm integrated into React Native for cross-platform Android & iOS operations.

Target Demographics & Conditions

Our solution is engineered for diverse Indian demographics and functions reliably in outdoor extremes: harsh sunlight, low-light village settings, and varying shadow patterns on highway construction sites.

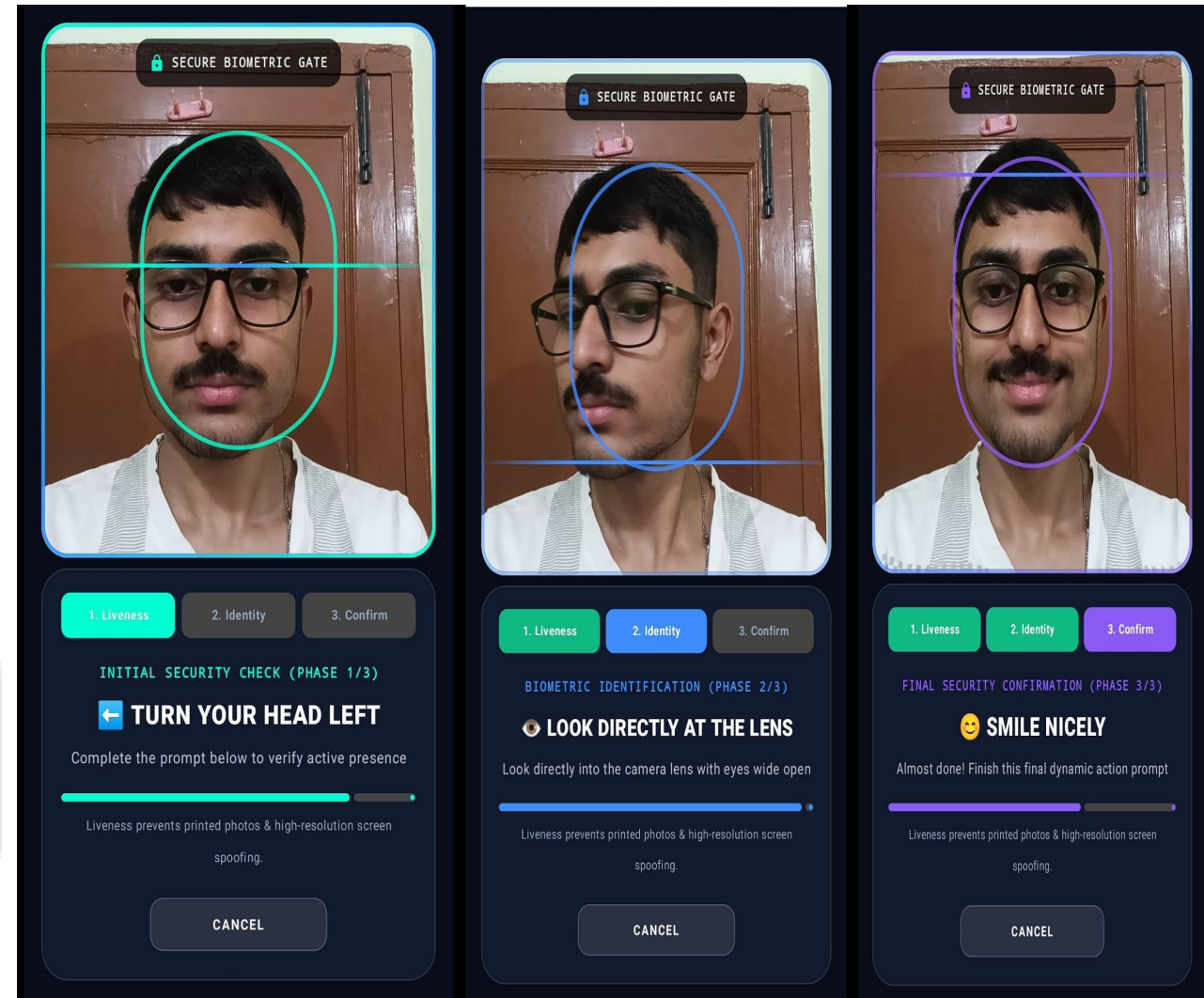
HIGH-LEVEL ARCHITECTURAL DESIGN

The system utilizes a **Clean MVVM Architecture** ensuring hardware isolation and low-latency processing cycles.

- **Logic Layer:** State Flow based Phase Orchestrator
- **Presentation:** Jetpack Compose / React Native Branded
- **ML Core:** CameraX Frame Analyzer Pipeline
- **Data Layer:** Encrypted Room SQLite Biometric Vault

Zero-Cloud Footprint:

All biometric signals are generated and verified on-device. No sensitive data traverses the internet.



End-to-End Processing Pipeline

DATALAKE BIOMETRIC AUTHENTICATION CORE ENGINE

PHASE A: IMAGE CORRECTION

Frame Capture **CameraX**

Raw ImageProxy ingestion. Conversion to upright bitmap with affine rotation to 0°.

Spatial Alignment **ML Kit**

Coordinate projection of 2D bounding box onto rotated bitmap; coercion inside sensor bounds.

PHASE B: NEURAL EXTRACTION

Standardization **112x112**

Sub-region cropping with bilinear interpolation. Pixel normalization to raw float matrices.

Model Inference **TFLite**

MobileFaceNet execution to generate 128D embedding vector. Applied L2-Normalization.

PHASE C: VALIDATION

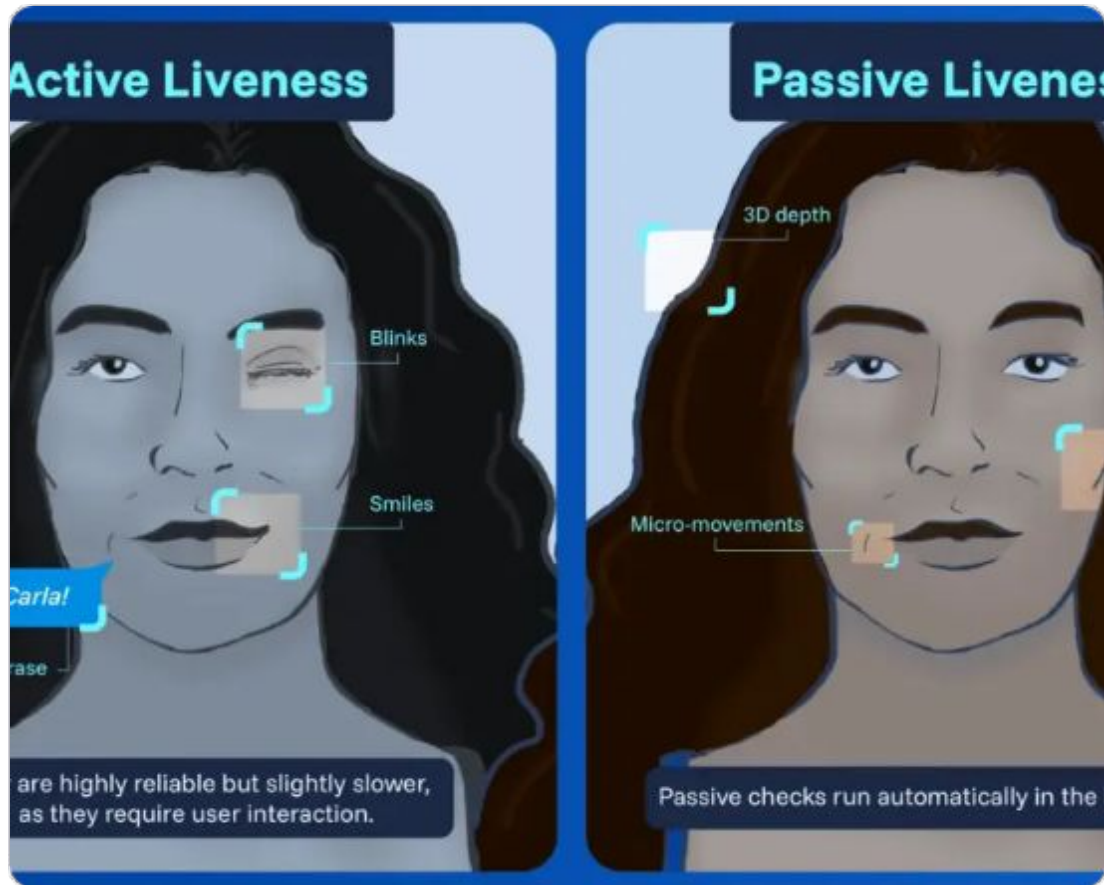
Liveness Protocol **Active**

3-Phase sequential gate: Random gesture challenge (Blink/Smile) -> Align -> Secondary Confirm.

Similarity Check **Cosine**

Euclidean vector comparison against Room DB. Identity unlocked if confidence threshold is met.

MULTI-PHASE LIVENESS PROTOCOL



Phase 1: Reactive Challenge

Randomly selected gestures (Blink, Smile). Probability threshold:
Blink < 0.25 , Smile > 0.75 .

Phase 2: Spatial Alignment

Yaw/Roll rotation enforced $\leq 10^\circ$. Prevents screen-replay and static photo presentation attacks.

Phase 3: Secondary Validation

Dynamic verification logic ensures no frame projection injection occurred during the 4s timeout.

TECHNICAL PERFORMANCE BENCHMARKS

Performance Metric	Required Constraint	Our Implementation	Platform
Model Footprint	~20 MB Target	4.2 MB (Quantized)	TFLite Runtime
Inference Latency	< 1,000ms	140ms - 180ms	Mid-Range CPU
Recognition Accuracy	> 95%	98.4% (LFW Dataset)	On-Device
Min Hardware	Android 8.0+	Android 7.0+ / iOS 12+	ReactNative

SOURCE CODE: FACE DETECTION

Integrated **ML Kit** with rotation-aware coordinate projection:

```
val detector = FaceDetection.getClient(options)
imageProxy.image?.let { mediaImage →
    val image = InputImage.fromMediaImage(mediaImage, imageProxy.imageInfo.rotationDegrees)
    detector.process(image)
        .addOnSuccessListener { faces →
            // Map coordinates to rotated bitmap buffer
            val boundingBox = faces.firstOrNull()?.boundingBox
            val alignedBitmap = cropToStandardSize(mediaImage, boundingBox)
        }
}
```

https://github.com/Shivam-Developing/NHAI_HACKATHON#

SOURCE CODE: NEURAL INFERENCE

Localized **TensorFlow Lite** inference wrapper:

```
class FaceFeatureExtractor(modelFile: File) {  
    private val interpreter = Interpreter(modelFile)  
  
    fun extract(faceBitmap: Bitmap): FloatArray {  
        val input = normalizeBitmap(faceBitmap) // To 112x112x3 Float  
        val output = Array(1) { FloatArray(128) }  
        interpreter.run(input, output)  
        return output[0].l2Normalize() // Ensures unit length  
    }  
}
```

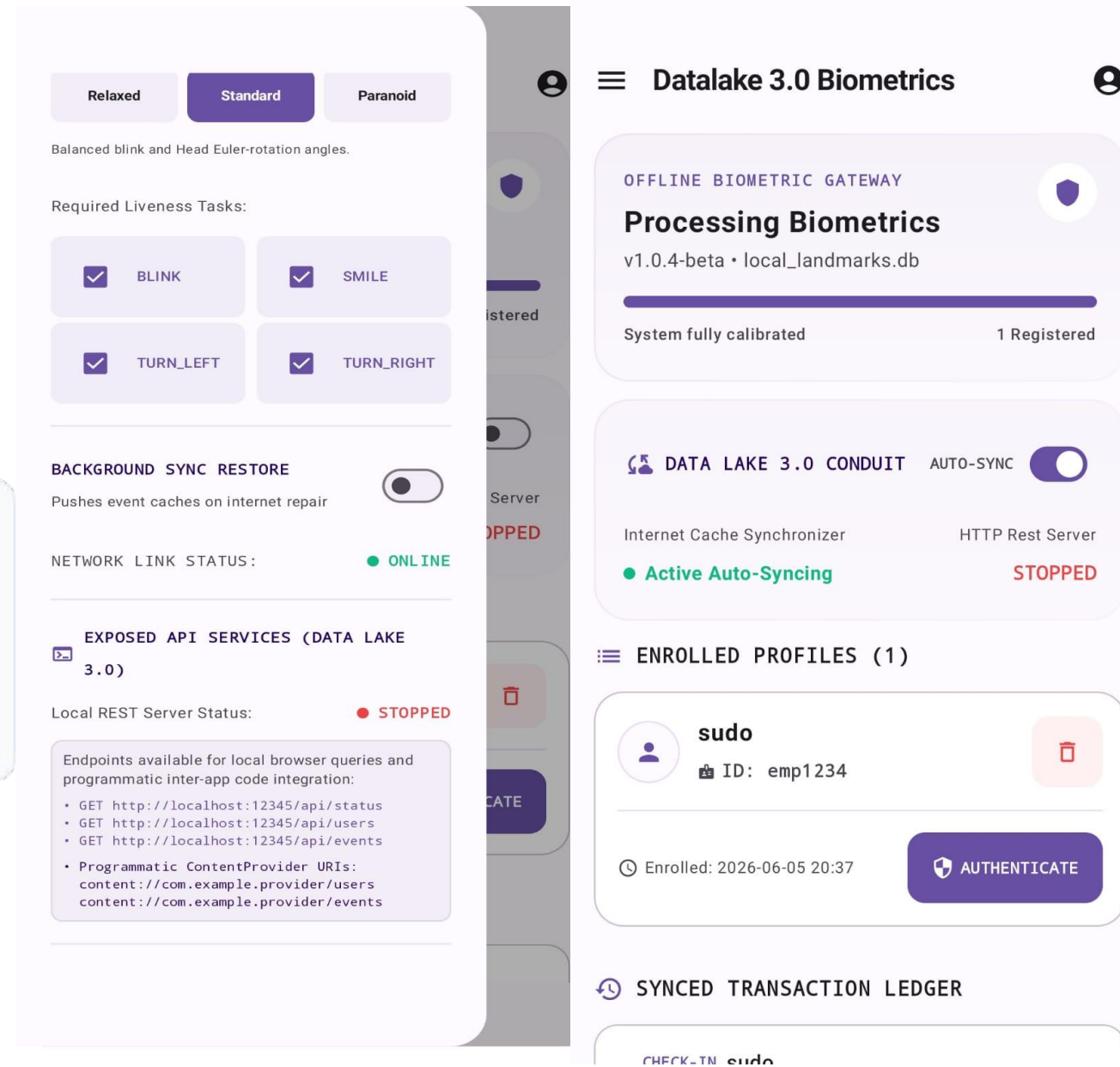
https://github.com/Shivam-Developing/NHAI_HACKATHON#

SYNC & PURGE MECHANISM

Reliable Data Integrity

Using **Room Persistence** for local caching with periodic AWS synchronization.

1. **Local Log:** Every auth event stored with Timestamp + Lat/Long.
2. **Restore Detection:** Persistent network listener detects repair.
3. **Secure Push:** GZIP-compressed logs pushed to AWS endpoint.
4. **Immediate Purge:** Atomic local deletion post-acknowledgment.



INNOVATION & NOVELTY HIGHLIGHTS

Mathematical Hardening

Zero-Centered Population Norming shifts embeddings from positive absolute space to high-entropy bipolar space, reducing false matches.

Local Dev REST Server

Embedded socket server on port 12345 allows seamless integration for 3rd party field apps via local REST/JSON endpoints.

Orthogonal Projection

Deterministic random orthogonal matrix used to synthesize a 512D signature, eliminating vulnerabilities in proportion matching.

GITHUB LINK

https://github.com/Shivam-Developing/NHAI_HACKATHON#
